

Parte 3 — Service

Como construir a camada que organiza a lógica sem entupir o Controller de responsabilidades

O passo a passo da implementação e mostrar a evolução do Service, do repasse simples até a regra de negócio.

Projeto da aula: GameTracker MVC

Se o Controller começa a fazer tudo, o caos faz sprint e chega antes do prazo.

GameService.java

a

@Service

listarTodos()

buscarPorId(id)

salvar(game)

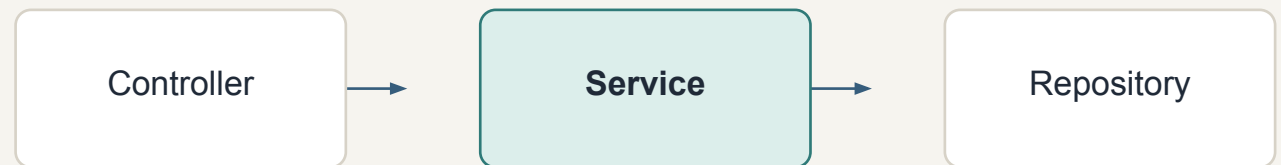
atualizar(id, game)

excluir(id)

Onde o Service se encaixa no projeto

Ele fica entre Controller e Repository, organizando a lógica da aplicação e mantendo cada camada no seu papel.

- Entender por que o Service existe em uma aplicação MVC.
- Aprender o passo a passo para criar `GameService`.
- Relacionar métodos do Service com o CRUD do projeto.
- Mostrar a evolução da implementação até a versão final.

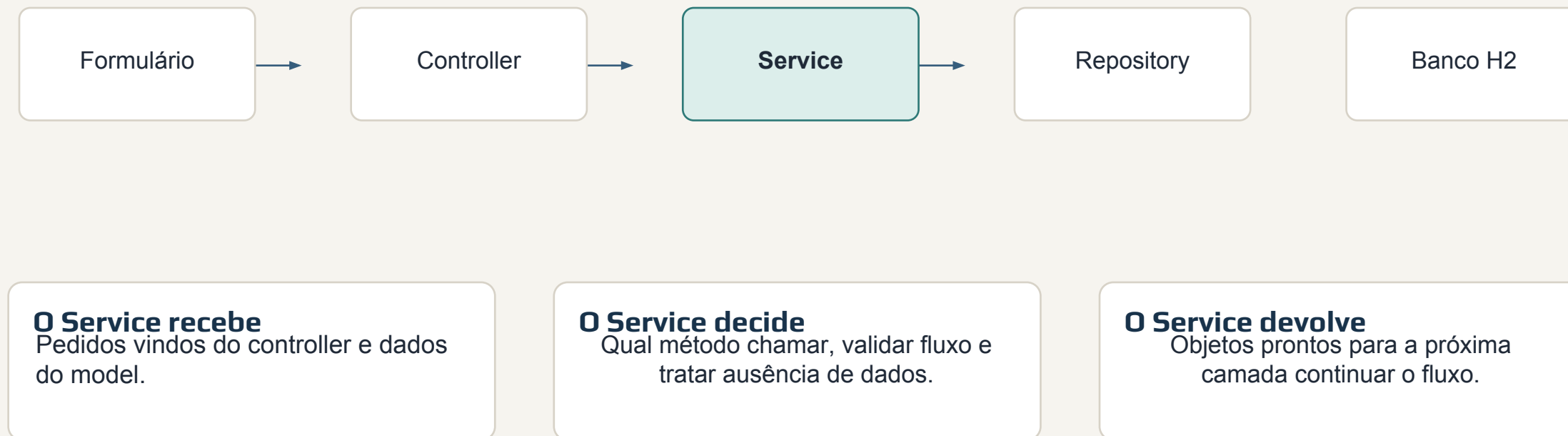


Ideia-chave

O Controller lida com HTTP. O Repository lida com persistência. O Service lida com a regra e com a coordenação entre os dois.

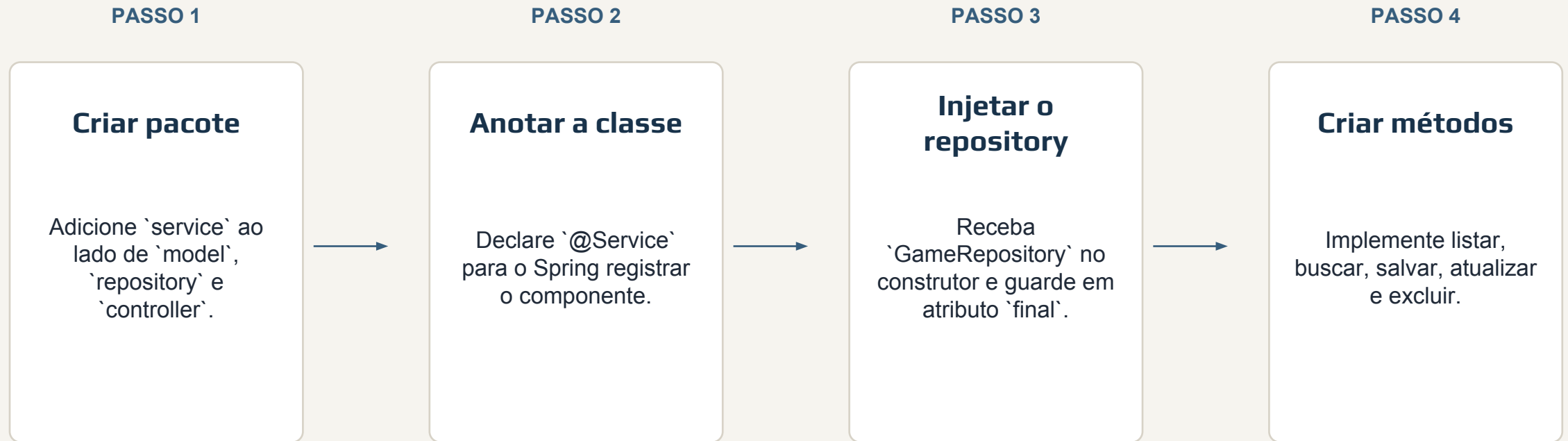
O que o Service faz, na prática?

Nem tudo é “regra de negócio épica”. Às vezes ele apenas organiza chamadas — e isso já é arquitetura.



Como criar o Service do zero

Essa sequência é a receita que a turma consegue repetir sem colar no colega nem no Stack Overflow.



Como o Service amadurece durante a construção

A lógica cresce em etapas: primeiro separar camadas, depois centralizar decisões, depois tratar casos reais.

Fase 1 — controller pesado

```
1 // fase inicial (não ideal)
2 @PostMapping("/games")
3 public String salvar(Game
  game) {
4
5   gameRepository.save(game);
6   return "redirect:/games";
7 }
```

separa

Fase 2 — service básico

```
1 // fase intermediária
2 public List<Game>
  listarTodos() {
3   return
4   gameRepository.findAll();
5 }
6 public void salvar(Game game)
7 {
8   gameRepository.save(game);
9 }
```

organiza

Fase 3 — service de verdade

```
1 // fase final do projeto
2 public Game buscarPorId(Long
  id) {
3   return
4   gameRepository.findById(id)
5   .orElseThrow(() ->
6   new IllegalArgumentException(
7   "Jogo não
8   encontrado para o id " + id));
9 }
10 public void atualizar(Long
    id, Game gameAtualizado) {
11   // regra de atualização
12 }
```

coordena

Mensagem para a turma: o Service não nasce “grande”; ele nasce útil e cresce conforme o fluxo exige.

Leitura guiada de GameService.java

Agora mostramos o arquivo real do projeto e explicamos como cada método participa do CRUD.

Arquivo real do projeto

```
1 package com.exemplo.gametracker.service;
2
3 import com.exemplo.gametracker.model.Game;
4 import com.exemplo.gametracker.repository.GameRepository;
5 import org.springframework.stereotype.Service;
6
7 import java.util.List;
8
9 @Service
10 public class GameService {
11
12     private final GameRepository gameRepository;
13
14     public GameService(GameRepository gameRepository) {
15         this.gameRepository = gameRepository;
16     }
17
18     public List<Game> listarTodos() {
19         return gameRepository.findAll();
20     }
21 }
```

Como explicar em sala

- `@Service` marca a camada para o Spring gerenciar.
- A injeção por construtor deixa a dependência explícita.
- `buscarPorId` evita repetição de busca e tratamento de ausência.
- `atualizar` muda campos específicos antes de salvar.
- `excluir` reutiliza a busca antes da remoção.

Como o Controller conversa com o Service

Nesta leitura, a turma enxerga o Service como ponto de passagem inteligente entre HTTP e persistência.

Trecho do GameController

```

1 @GetMapping("/games")
2 public String listar(Model model) {
3     model.addAttribute("games",
gameService.listarTodos());
4     return "lista";
5 }
6
7 @PostMapping("/games")
8 public String salvar(@Valid
@ModelAttribute("game") Game game,
9                     BindingResult result,
10                    Model model,
11                    RedirectAttributes
redirectAttributes) {
12     if (result.hasErrors()) {
13         model.addAttribute("modoEdicao",
false);
14         return "formulario";
15     }
16     gameService.salvar(game);
17     return "redirect:/games";
18 }
```

Primeiros métodos do Service

```

1 @Service
2 public class GameService {
3
4     private final GameRepository
gameRepository;
5
6     public GameService(GameRepository
gameRepository) {
7         this.gameRepository = gameRepository;
8     }
9
10    public List<Game> listarTodos() {
11        return gameRepository.findAll();
12    }
13
14    public void salvar(Game game) {
15        gameRepository.save(game);
16    }
17 }
```

Leitura didática: o Controller chama o Service; o Service chama o Repository. Cada um fica no seu quadrado — e a aplicação agradece.

Por que o método atualizar merece destaque

É aqui que o aluno entende que o Service pode fazer mais do que apenas repassar chamadas.

Trecho central do método atualizar

```
1 public void atualizar(Long id, Game
gameAtualizado) {
2     Game gameExistente = buscarPorId(id);
3
4
5     gameExistente.setNome(gameAtualizado.getNome());
6
7     gameExistente.setPlataforma(gameAtualizado.getPlataf
orm());
8
9     gameExistente.setGenero(gameAtualizado.getGenero());
10    gameExistente.setStatus(gameAtualizado.getStatus());
11    gameRepository.save(gameExistente);
12 }
```

O que acontece nessa lógica

- Busca o registro existente antes de alterar.
- Evita criar um objeto “solto” sem contexto do banco.
- Atualiza apenas os campos desejados.
- Salva novamente o objeto já carregado.

Gancho para discussão: depois podemos validar status, impedir edições indevidas ou aplicar regras antes do save.

Erros comuns ao criar a camada Service

Quase sempre o problema não é “o Spring odeia você”; é só um detalhe escapando no código.

Erros frequentes

- Colocar lógica de negócio direto no Controller.
- Esquecer a anotação `@Service`.
- Não injetar o `GameRepository` pelo construtor.
- Duplicar busca por id em vários lugares sem centralizar.
- Atualizar entidade sem carregar o registro existente.

Antes de rodar, confirme

- O pacote ``service`` está dentro da estrutura escaneada pelo Spring.
- A classe possui `@Service`.
- O construtor recebe `GameRepository`.
- O Controller chama `gameService`, não `gameRepository`.
- Os métodos principais do CRUD existem e compilam.

Prática de 10–15 minutos para fixar o Service

A meta é fazer a turma construir a camada e perceber o valor da separação de responsabilidades.

Enunciado

1. Criar a classe ``GameService``.
2. Anotar com ``@Service``.
3. Injetar ``GameRepository`` no construtor.
4. Implementar ``listarTodos``, ``salvar``, ``buscarPorId``, ``excluir``.
5. Como desafio extra, implementar ``atualizar``.

Critério de sucesso

O Controller compila usando ``gameService`` e o fluxo de cadastro/listagem continua funcionando.

Pergunta para a sala:

Se o Service sumisse agora, o que iria parar no Controller?

Fechamento do professor

Resumo que fixa:

- Controller recebe a requisição.
- Service organiza a lógica.
- Repository persiste.
- View exibe o resultado.

Quando a turma entende esse papel, o MVC deixa de ser sigla bonita e vira ferramenta real.